



5. Architectural Elements

Objetivo

Antes de definir motores, componentes y límites del sistema, es necesario establecer un lenguaje arquitectónico común.

Este documento define los elementos fundamentales que componen la arquitectura conceptual del Personal Operating System (POS), su propósito, nivel de abstracción y relación entre ellos.

Su objetivo es evitar ambigüedades entre conceptos como *Engine*, *Component*, *Process* o *State*, proporcionando una estructura consistente sobre la cual construir el resto de la arquitectura.

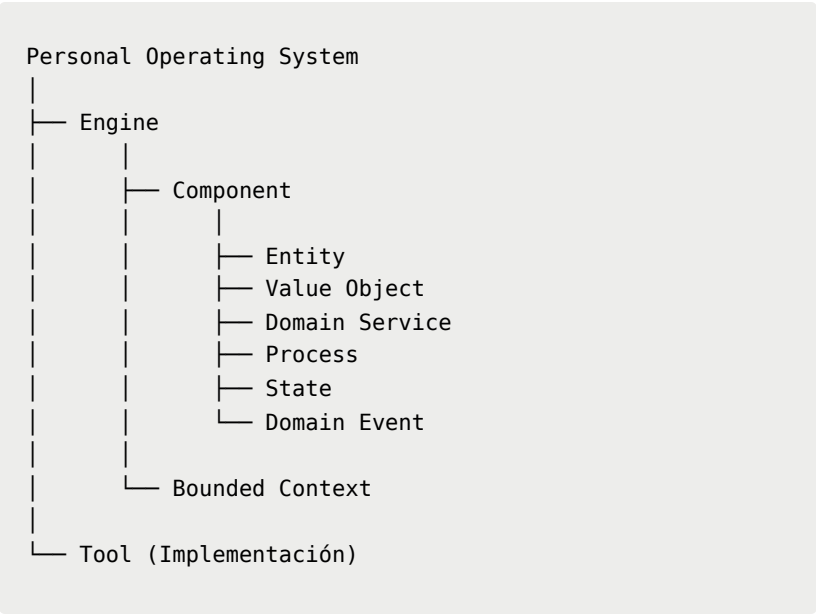


Principios

Los elementos arquitectónicos del POS cumplen los siguientes principios:

- Cada nivel posee una única responsabilidad.
- Cada elemento existe en un único nivel de abstracción.
- Ningún elemento reemplaza la responsabilidad de otro.
- Los procesos transforman entidades; nunca las representan.
- Los estados describen entidades; nunca procesos.
- La implementación tecnológica no modifica esta estructura.

Jerarquía Arquitectónica



Niveles Arquitectónicos

Nivel	Propósito	Ejemplo
System	Arquitectura completa	Personal Operating System
Engine	Gran capacidad del sistema	Operations Engine
Component	Módulo funcional dentro de un Bounded Context	Task Management
Entity	Objeto del dominio con identidad	Task
Value Object	Información sin identidad propia	Priority
Process	Transformación del dominio	Execute
State	Condición actual de una entidad	Planned
Domain Service	Lógica de negocio compartida	Prioritization
Domain Event	Cambio relevante ocurrido en el dominio	Task Completed
Tool	Implementación física	Notion, Obsidian, Google Drive

1. System

Definición

El **System** representa la totalidad del Personal Operating System.

Define la arquitectura conceptual completa, sus principios, lenguaje, reglas y relaciones.

Todo elemento del dominio pertenece al System.

Responsabilidades

- Definir la arquitectura global.
- Mantener coherencia conceptual.
- Integrar todos los Engines.

No es

- una aplicación;
- una herramienta;
- un repositorio de información.

2. Engine

Definición

Un **Engine** representa una gran capacidad permanente del sistema.

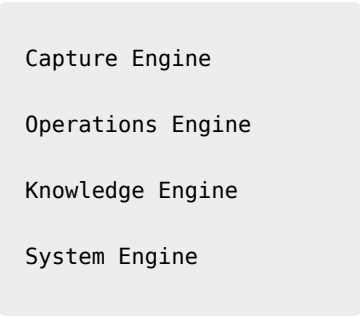
Cada Engine agrupa responsabilidades relacionadas dentro de un mismo dominio funcional.

Los Engines constituyen el nivel superior de organización del sistema.

Responsabilidades

- Agrupar capacidades relacionadas.
- Definir límites funcionales de alto nivel.
- Agrupar uno o más Bounded Contexts.

Ejemplos



No es

- un proceso;
- un workflow;
- una entidad.

Un Engine puede contener uno o más Bounded Contexts.

Los Bounded Contexts representan los límites

3. Component

Definición

Un **Component** es un módulo funcional perteneciente a un único Bounded Context.

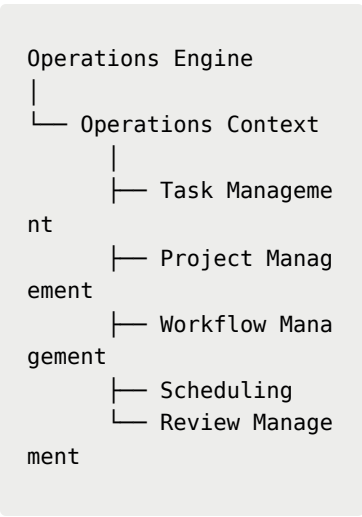
Representa una capacidad concreta que puede evolucionar de manera relativamente independiente.

Responsabilidades

- Resolver un único problema del dominio.
- Encapsular reglas relacionadas.

Exponer capacidades al resto del Bounded Context.

Ejemplo



Un Component puede contener

- Entities
- Domain Services
- Processes
- Events
- Rules

No es

4. Entity

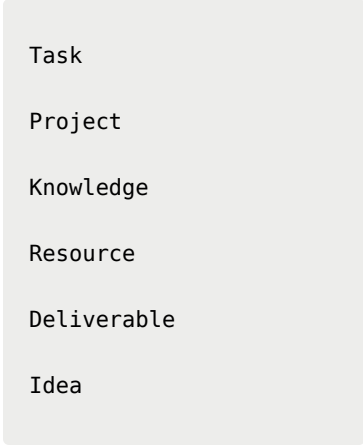
Definición

Una **Entity** representa un elemento del dominio con identidad propia y ciclo de vida.

Su identidad permanece aunque cambien sus atributos.

Las entidades constituyen el núcleo del modelo de dominio. Esta definición es consistente con el Ubiquitous Language ya establecido.

Ejemplos



semánticos del dominio.

Los Components representan capacidades funcionales dentro de dichos Contexts.

- un estado;
- un paso del workflow;
- una carpeta.

5. Value Object

Definición

Un **Value Object** representa información que describe una Entity pero que no posee identidad propia.

Dos Value Objects con el mismo valor son equivalentes.

Ejemplos

- Priority
- Due Date
- Status
- Tag
- Version

6. Process

Definición

Un **Process** describe una transformación del dominio.

No representa módulos ni funcionalidades del sistema.

Representa cambios sobre entidades.

Esta definición mantiene la separación establecida previamente entre entidades, procesos y estados.

Ejemplos

- Capture
- Process
- Execute
- Learn
- Build
- Deliver
- Review
- Archive

Responsabilidades

Transformar entidades.

Nunca almacenarlas.

Nunca representarlas.

7. State

Definición

Un **State** representa la condición actual de una Entity dentro de su ciclo de vida.

Los estados no contienen comportamiento.

Son consecuencia de la ejecución de Processes.

Ejemplos

- Captured
- Processed
- Planned
- In Progress
- Completed
- Archived

8. Domain Service

Definición

Un **Domain Service** encapsula lógica del dominio que no pertenece naturalmente a una única Entity.

Se utiliza cuando una regla involucra múltiples entidades o requiere coordinación.

Ejemplos

- Prioritization
- Scheduling
- Knowledge Classification
- Resource Allocation

9. Domain Event

Definición

10. Tool

Definición

Un **Domain Event** representa un hecho relevante ocurrido dentro del dominio.

Los eventos son inmutables y describen algo que ya sucedió.

Ejemplos

- Task Created
- Task Completed
- Knowledge Validated
- Deliverable Delivered
- Asset Published

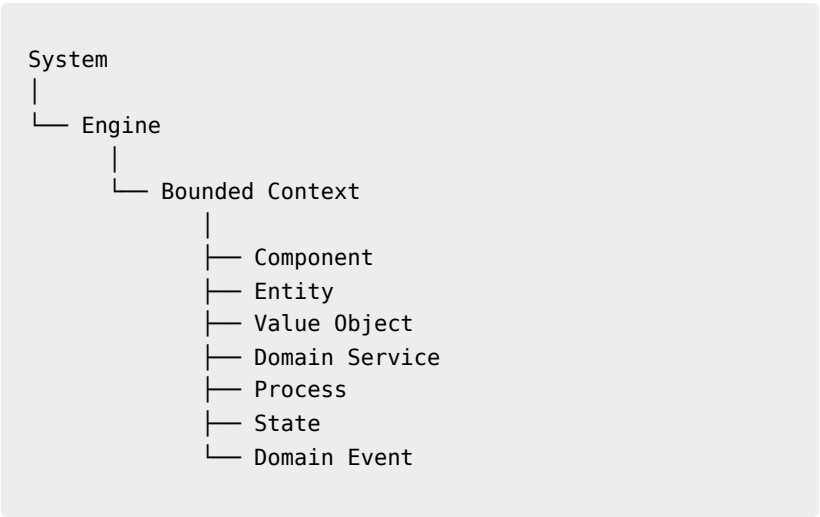
Una **Tool** es la implementación concreta del sistema.

Las herramientas ejecutan la arquitectura, pero nunca la definen, en línea con el principio *Tool Agnostic* de Foundation.

Ejemplos

- Notion
- Obsidian
- Google Drive
- ChatGPT
- VS Code

Relaciones entre Elementos



Reglas Arquitectónicas

AR-001 — Jerarquía estricta

Cada elemento pertenece a un único nivel arquitectónico.

AR-002 — Responsabilidad única

Un elemento no puede representar simultáneamente un Engine, Component y Process.

AR-003 — Separación entre estructura y comportamiento

- **Components** organizan capacidades.
- **Entities** representan el dominio.
- **Processes** producen transformaciones.
- **States** representan condiciones.
- **Events** representan hechos ocurridos.

AR-004 — Independencia tecnológica

Ningún elemento arquitectónico depende de una herramienta específica.

AR-005 — Evolución controlada

Los Engines pueden incorporar nuevos Bounded Contexts.

Los Bounded Contexts pueden incorporar nuevos Components.

Los Components pueden evolucionar sin modificar el modelo conceptual del dominio.



Resultado esperado

Una vez definidos los **Architectural Elements**, el POS dispone de una taxonomía arquitectónica estable que separa claramente:

- **qué existe** (Entities),
- **qué organiza** (Components),
- **qué transforma** (Processes),
- **qué estado posee** (States),
- **qué sucede** (Domain Events),
- **qué coordina** (Domain Services),
- **dónde se agrupa** (Engines),
- **dónde se delimita semánticamente** (Bounded Contexts)
- **y cómo se implementa** (Tools).

Esta separación reduce la ambigüedad conceptual y proporciona una base sólida para definir los **Bounded Contexts**, el **Information Flow** y el **Conceptual Data Model** sin mezclar niveles de abstracción.